EURS-Net V2: Team Pre-Flight Engineering Report

Comprehensive Status Summary & Immediate RTX 5090 Execution Plan

Project: Embedded Underwater Restoration & Segmentation (EURS) — **Venue:** MaCVi @ WACV 2027
Team Members: Shubham (Data & Deployment Lead) Tanmay (Modeling & GPU Lead) Arin (Tooling & Quality Lead)
Target Hardware: NVIDIA Jetson AGX Orin (Track 2) **Compute Cluster:** NVIDIA RTX 5090 (Blackwell 32GB)
Date: October 2026 — **Status:** Pre-Flight Invariants 100% Verified (Zero Known Defects)

---

1 Executive Summary & Competition Constraints

The **EURS Challenge** addresses simultaneous underwater color restoration and 5-class semantic segmentation (Background, Coral, Fish, Elasmobranchii/Sharks, Other). Solutions must satisfy strict hardware deployment caps statically verified prior to Jetson AGX Orin evaluation:

Parameter Cap: $\leq 1,000,000$ parameters (EURS-Net V2 achieved: **150,438 parameters**, $\sim 5\%$*underbudget*).**Compu**
12.0 GFLOPs at inference resolution (EURS-Net V2 achieved: **4.47 GFLOPs**, $\sim 3\%$*underbudget*).

Scoring Contract: Composite score $= 0.60\ \mathrm{Q}_{\mathrm{seg}} + 0.30 Q_{\mathrm{rest}} + 0.10 Q_{\mathrm{NR}}$ with severe coverage penalties for missing rare classes.

2 Pre-Flight Work Accomplished (Before GPU Deployment)

To guarantee zero wasted compute hours, runtime exceptions, or silent algorithmic defects on the RTX 5090 cluster, all three team members completed rigorous modular preparation:

2.1 1. Tanmay Das (Model Architecture, Loss Formulations & Training Engine)

EURS-Net V2 Architecture: Implemented a 150K-parameter dual-task network in `models/`:

- *Pre-Net:* Fast White-Balance Residual (6 learnable affine parameters) correcting underwater green/blue gain attenuation.
- *Shared Encoder:* MobileNetV2 inverted bottlenecks with horizontal/vertical Coordinate Attention (CA) across Stages 1–4.
- *Dual Decoders:* Progressive Restoration Decoder ($28 \times 128 \rightarrow 1024 \times 1024$ residual ΔI map) and Progressive Feature Pyramid Network (FPN) Segmentation Decoder with Small-Object High-Resolution Neck.

Multi-Task Loss Objective (`losses/`): Combined Mean-1.0 Square-Root Smoothed Class-Weighted Focal Loss + Soft Dice Loss ($\mathcal{L}_{\mathrm{seg}}$) with Charbonnier + MS-SSIM + Laplacian Edge Sharpness Loss ($\mathcal{L}_{\mathrm{rest}}$), statically balanced at $.0\mathrm{L}_{\mathrm{seg}} + 0.5\mathcal{L}_{\mathrm{rest}}$.

20-Suite Pre-Flight Testing Engine (`tests/`): Authored 12 Phase 1 suites, 8 Phase 2A gatekeeper tests, and a 14-point heavy edge-case stress audit covering black/white inputs, odd batch sizes, gradient accumulation boundaries, and worker multiprocessing.

Small-Sample & Overfit Sanity Audits:

- *20-Image Prototype Split:* Tested pipelines locally on a toy `mini_dev` dataset before GPU access.
- *10-Sample Memorization Test (`test_overfit.py`):* Proved mathematical convergence ($> 50\%$ loss reduction) and full class coverage on a micro-batch of real images.
- *1-Epoch Debug Loop (`train_debug.py`):* Validated step semantics, forward/backward loops, and optimizer steps on CPU.

RTX 5090 Blackwell Hardening: Centralized paths via `utils/paths.py`, enabled native `bfloat16` mixed precision (disabling `GradScaler` for BF16), eliminated strict CUDA bilinear determinism crashes, and authored `configs/baseline_v1.yaml`.

2.2 2. Shubham (Data Engineering, Scientific Validation Protocol, Evaluation & Deployment)

Scientific 3-Tier Validation Protocol (`data/splits.json`): Discovered severe dataset scene-sparsity (Class 3 sharks present in only 2 scenes; Class 4 in 3 scenes). Replaced naive random splitting with an audited, non-leaking protocol:

- *Tier 1 (Canonical 16/4 Split):* Balances rare classes across Train and Val with 0% data leakage. Val: `fish1, fish12, seahorse1, sharkFish`. Train: `ray1, starfish2, turtles1` + 13 common scenes.
- *Tier 2 (Rare-Class Swap):* 2-run protocol swapping rare scene assignments for major architectural decisions.
- *Tier 3 (4-Fold OOF Aggregation):* 4 folds of 15 train / 5 val scenes, aggregating complete out-of-fold predictions across all 20 scenes for release benchmarking.

Production Data Pipeline Specification (`data/`): Designed synchronized triplets loader (`degraded`, `clean`, `mask`) with lockstep spatial augmentations (horizontal flips, spatial crops) while strictly outlawing uncalibrated color jitter to protect white-balance learning.

Evaluation Engine (`metrics/`, `evaluate.py`): Implemented official Track 2 metrics: confusion matrix mIoU, Macro , $NormalizedPSNR$(5–35 dB window), SSIM, Laplacian sharpness ratio, and dual-lens reporting (**Global Score + Scene-Macro Diagnostic Scorecard**).
Static ONNX Deployment Contract (`deploy/`): Authored static shape exporter (1024, $Opset18)forJetsonOrinwith$ 10^{-4} numerical parity against ONNX Runtime, preserving modeling freedom for experimental resolutions (84 $\times$768 vs 024 $\times$1024).

2.3 3. Arin (Tooling, Quality Assurance, Visualization & Submission Packaging)

Exploratory Data Analysis (`tools/eda_stats.py`): Profiled pixel mass across all 5 classes, identified extreme background domination ($> 90\%$), and cataloged high-resolution camera aspect ratios (000 $\times$3000 to 710 $\times$7283).
Qualitative Diagnostic Visualizer (`tools/visualizer.py`): Built 3-panel and 4-panel visual tools rendering degraded images, clean references, masks, and model prediction overlays for error inspection and academic paper figures.
Track 1 Submission Packager (`tools/make_submission_track1.py`): Automated generation of `submission_track1.`

- Enforces exact directory hierarchy: root-level `restored/` and `seg_masks/`.
- Guarantees 3-channel RGB PNGs for restoration and strictly 1-channel `uint8` PNGs for semantic masks.
- Wraps `resize_for_submission.py` to upscale model outputs back to original dimensions using bicubic interpolation for RGB and nearest-neighbor for masks.

Experiment Telemetry (`experiments/ablations.md`): Structured tracking tables recording experiment IDs, parameter counts, GFLOPs, PSNR, SSIM, and mIoU.

3 Modular Work Division Matrix

Functional Domain	Tanmay (Modeling & Training)	Shubham (Data & Deployment)	Arin (Tooling & Packaging)
			Neural Architecture
EURS-Net V2 (FWB + CA + Decoders)	Budget audits & FLOP compliance	Records parameter logs **Loss Formulations**	Focal, Soft Dice, Charbonnier, MS-SSIM
Evaluates loss weighting impact	Tracks loss convergence curves **Dataset Management**	Consumes DataLoader batches	3-Tier Split Manifests & Pipeline
EDA pixel counts & class ratios **Testing & Auditing**	20 Automated verification suites	Overfit test harness & parity checks	Visual prediction comparisons **Evaluation Metrics**
Calls eval hooks during training	Computes Global & Scene-Macro scores	Logs leaderboard benchmarks **Final Deployment**	Produces trained weights (`.pt`)
Exports static Track 2 ONNX	Packages Track 1 submission ZIP		

4 Immediate Action Steps Once Hands-On With RTX 5090

Upon accessing the RTX 5090 environment, the team will execute the following sequenced roadmap:

1. **Step 1: Environment & Hardware Capability Smoke Test** ($\sim 0s$) $VerifyPython3.10+, PyTorchwithCUDA12.8+, a$ `python tests/test_cuda_determinism.py` ***(Verifies 32GB VRAM, native BF16, finite gradients).***
2. **Step 2: Heavy Edge-Case & Concurrency Stress Audit** ($\sim 5s$) $Executethe14-pointedge-casesuitetoensurezerorun$ `python -m unittest tests/test_heavy_edge_cases.py`
3. **Step 3: Physical Dataset Extraction & Directory Harmonization** () $Extract$`eurs_train.zip`$tostoragedirectory.En$ `cd data && ln -s gt_images clean && ln -s gt_masks masks`
4. **Step 4: End-to-End Phase 2A Pre-Flight Gatekeeper** () $Runthecompletegatekeeperfromrepositoryroot$: `python tests/test_phase2_preflight.py` ***(100% PASS across all 8 work packages).***
5. **Step 5: Training Engine Micro Dry-Run** ($\sim 0s$) $Validatetheproductiontrainingengineusingthedry-runguardrail(executes2batchesand1validationbatch)$: `python train.py --config configs/baseline_v1.yaml --dry_r`
6. **Step 6: Launch Full Production Training Run (50 Epochs)** Launch background training with persistent logging: `python train.py --config configs/baseline_v1.yaml > train_baseline_v1.log 2>&1 &` *Config:* Resolution 024 $\times$1024, Batch Size 4, Grad Accum 4 (Effective 16), `bfloat16`, 4 workers, SceneBalancedSampler.
7. **Step 7: Checkpoint Evaluation & Model Export (Post-Training)** Shubham evaluates `checkpoints/best_model.pt` and exports the frozen model to Track 2 ONNX: `python evaluate.py --checkpoint checkpoints/best_model.pt --device cuda python deploy/export_onnx.py --checkpoint checkpoints/best_model.pt`
8. **Step 8: Submission Packaging & Official Leaderboard Upload** Arin packages Track 1 predictions, runs resolution restoration, and generates the validated ZIP archive: `python tools/make_submission_track1.py --output submission_track1_v1.zip` *(Upload to WACV 2027 server).*